

# BC7215AC A/C Control Examples

---

The **BC7215AC Air Conditioner Remote Control Library** provides 5 example applications, each available in both English and Chinese versions:

- 
- ESP8266 Serial Monitor
- ESP32 Serial Monitor
- NANO 33 IoT Serial Monitor
- ESP32 LCD
- ESP32 MQTT

The **Serial Monitor version** is the simplest demo. It only requires connecting any ESP8266 or ESP32 Arduino development board to the BC7215A IR transceiver module, then using the Arduino IDE's built-in Serial Monitor as the human-machine interface to control the air conditioner.

The **LCD** and **MQTT** versions require the LilyGO TTGO T-Display ESP32 board, which comes with a built-in LCD display and two physical buttons. These hardware features allow the demo programs to run independently without a PC.

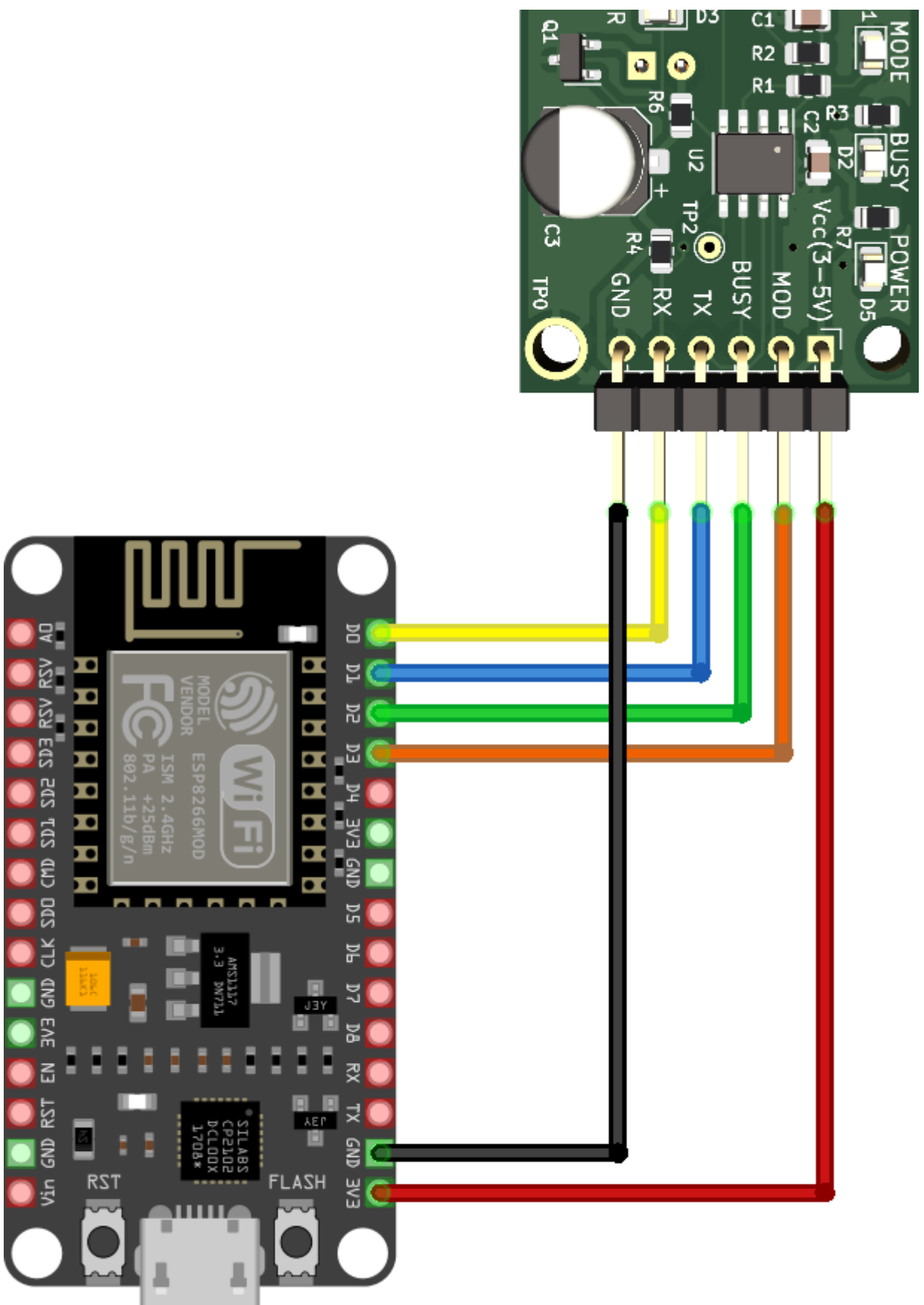
The **MQTT version** builds on the LCD version by adding MQTT networking support. This enables users to test network-based air conditioner control via a public MQTT broker. The MQTT version still retains local button control and can report the updated air conditioner state to the MQTT server.

The examples use ESP8266 NodeMCU boards and ESP32 TTGO T-Display boards. The hardware connections are as follows:

## ESP8266:

- GPIO5 → BC7215A TX
- GPIO16 → BC7215A RX
- GPIO0 → BC7215A MOD
- GPIO4 → BC7215A BUSY
- 3.3V → BC7215A VCC

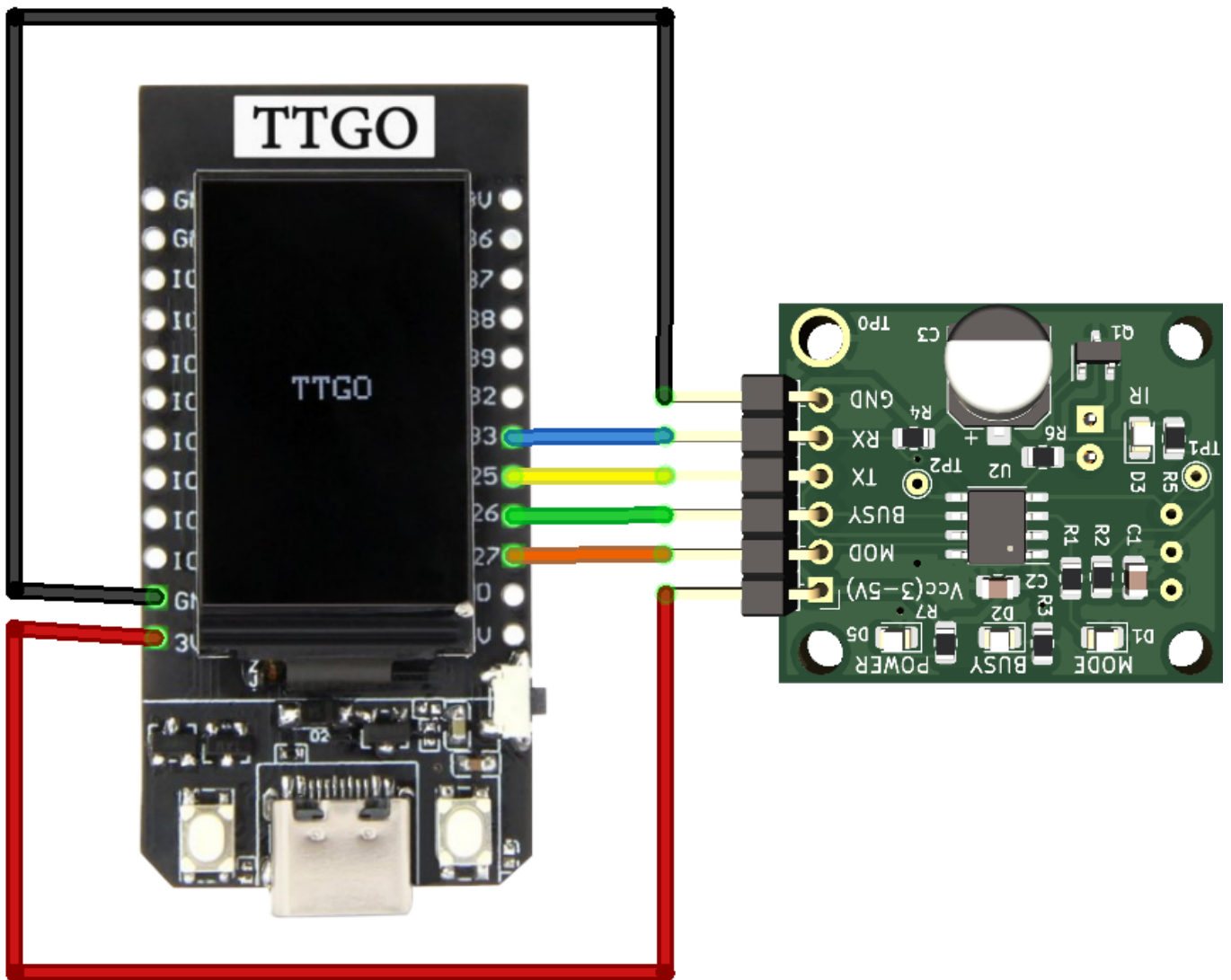




fritzing

## ESP32:

- GPIO25 → BC7215A TX
- GPIO33 → BC7215A RX
- GPIO27 → BC7215A MOD
- GPIO26 → BC7215A BUSY
- 3.3V → BC7215A VCC

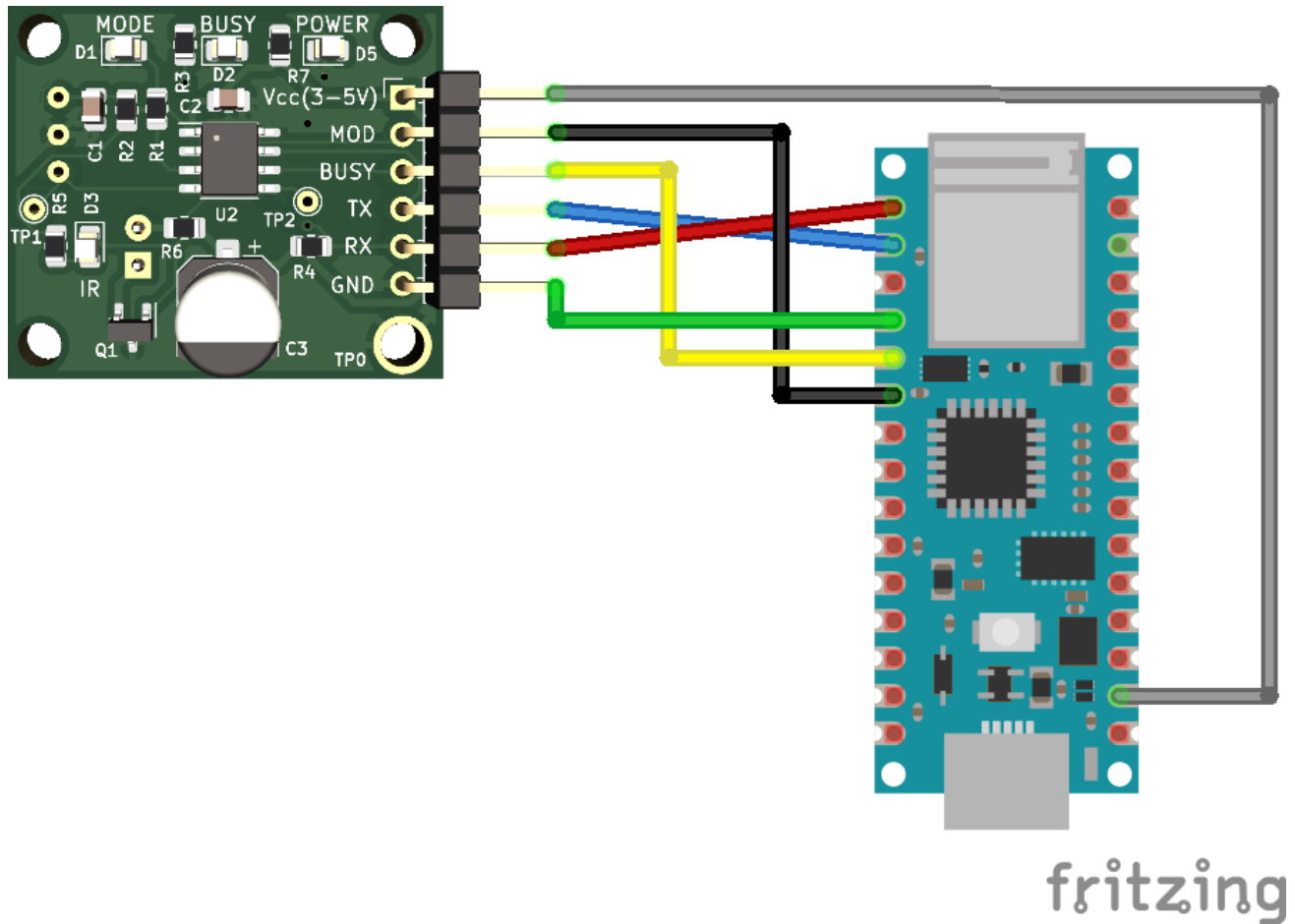


fritzing

## Arduino NANO33 IoT:

- RX - BC7215A TX
- TX - BC7215A RX

- D3 - BC7215A MOD
- D2 - BC7215A BUSY
- 3.3V - BC7215A VCC



## Serial Monitor Version

This version uses the Arduino IDE Serial Monitor (baud rate: 115200). Three program variants are provided:

- NANO 33 IoT
- ESP8266
- ESP32

The **serial monitor version** uses a state machine so tasks can proceed in parallel without getting stuck.

After uploading the program, the main menu appears in the Serial Monitor. If not, simply press Enter or reset the Arduino board.

```
/dev/ttyACM0
| Send

clk_drv:0x0bfff000,d_drv:0x00,cs0Efff0x00,hd_drv:0x00fff000}drv:0x00
mode'Tf 0ffffdiv:1
f+fff0x3fff0030,len'Sj
load:0x40078000,ff:16612
load:0x400804fbbfff3480
entry 0f00805b4

*****
* Welcome to BC7215A Universal AC Controller Demo *
*****
AC Library Version: V5.4 build 2604
Current AC control library status: Not initialized (must be paired with AC before use)
Please select:
  1. Pairing with AC
  2. Control air conditioner
  3. Save pairing data
  4. Read saved data and pair with it
  5. Try next match (if paired successfully but cannot control AC properly)
  6. Load predefined protocol
  7. Parse IR signal

Now performing IR AC pairing.
Please set AC remote to < Cooling mode, 25°C(77°F)>, then press any key to continue...
Now please aim at IR receiver and press <Fan Control> button on remote.
Will automatically proceed to next step after receiving signal...
Received data: 49 9 20 50 A 0 2 0 48 4A 0 81 13 0 0 20 10
AC control library initialization using received data **SUCCESS** !!!
Now please enter any content, program will return to main menu and AC control can begin...
```

☒ Autoscroll ☐ Show timestamp    Newline    115200 baud    Clear output

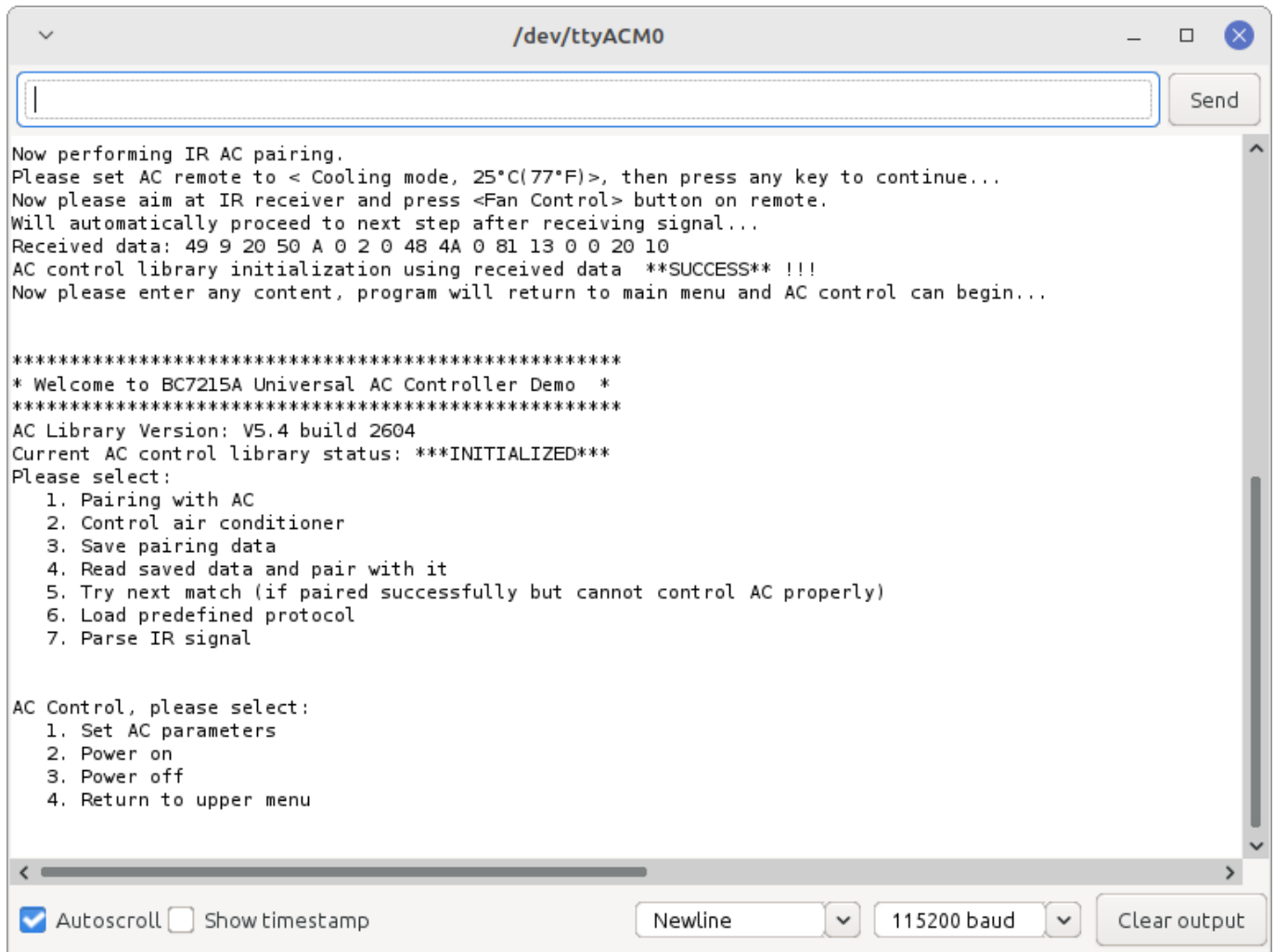
## Initialization

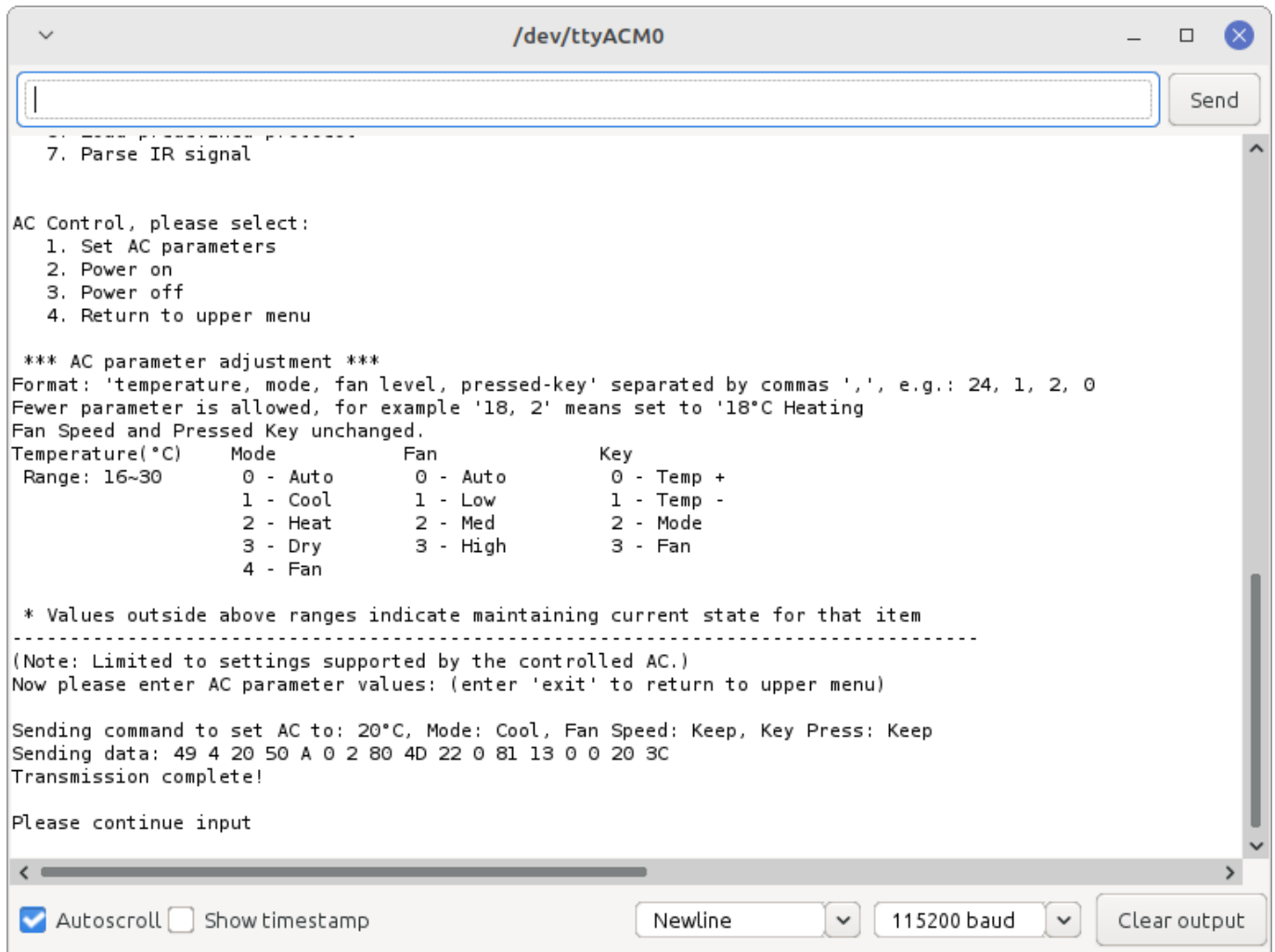
First-time use requires **Pairing With A/C**. The collected data is used to initialize the library. The process is step-by-step, guided on-screen. If pairing fails repeatedly, the AC model may be one of the rare types that BC7215A cannot directly decode. In this case, you can try using **predefined protocols** to test control.

## A/C Control

Once initialized successfully, you can start controlling the AC through a **two-level menu**:

1. Choose control type (parameters such as temperature, or power on/off).
2. Enter parameter values (temperature, mode, fan speed).





```
7. Parse IR signal

AC Control, please select:
1. Set AC parameters
2. Power on
3. Power off
4. Return to upper menu

*** AC parameter adjustment ***
Format: 'temperature, mode, fan level, pressed-key' separated by commas ',', e.g.: 24, 1, 2, 0
Fewer parameter is allowed, for example '18, 2' means set to '18°C Heating
Fan Speed and Pressed Key unchanged.
Temperature(°C)    Mode          Fan              Key
Range: 16~30      0 - Auto      0 - Auto          0 - Temp +
                  1 - Cool      1 - Low           1 - Temp -
                  2 - Heat      2 - Med           2 - Mode
                  3 - Dry       3 - High          3 - Fan
                  4 - Fan

* Values outside above ranges indicate maintaining current state for that item
-----
(Note: Limited to settings supported by the controlled AC.)
Now please enter AC parameter values: (enter 'exit' to return to upper menu)

Sending command to set AC to: 20°C, Mode: Cool, Fan Speed: Keep, Key Press: Keep
Sending data: 49 4 20 50 A 0 2 80 4D 22 0 81 13 0 0 20 3C
Transmission complete!

Please continue input
```

☒ Autoscroll ☐ Show timestamp    Newline    115200 baud    Clear output

## ESP32 LCD Version

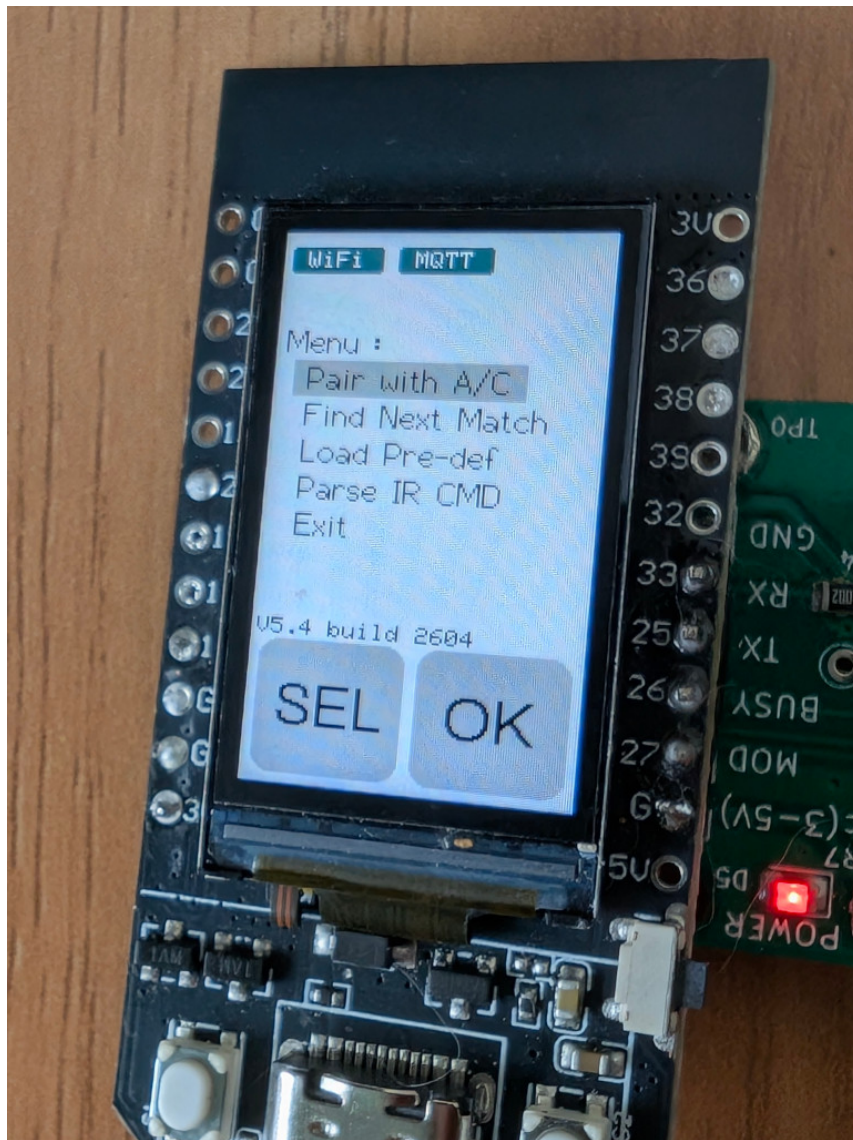
This version uses the TTGO T-Display Arduino board (135×240 ST7789 LCD). It relies on **Bodmer's TFT\_eSPI library**, installable via Arduino IDE Library Manager.

You must configure the library by replacing `User_Setup.h` in TFT\_eSPI with the one provided in the `extras/config` directory of the BC7215AC library.

When the program runs for the first time, the menu appears:

- Left button (**SEL**) selects items.
- Right button (**OK**) confirms.

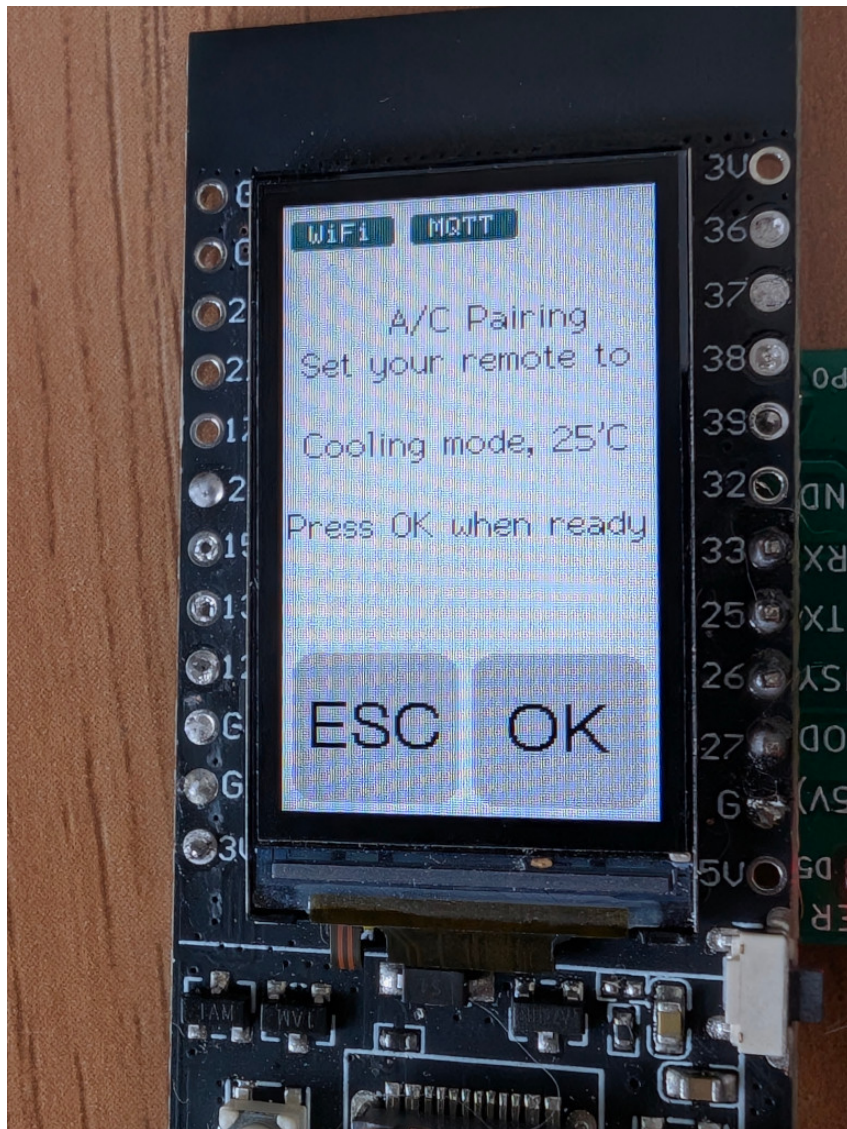




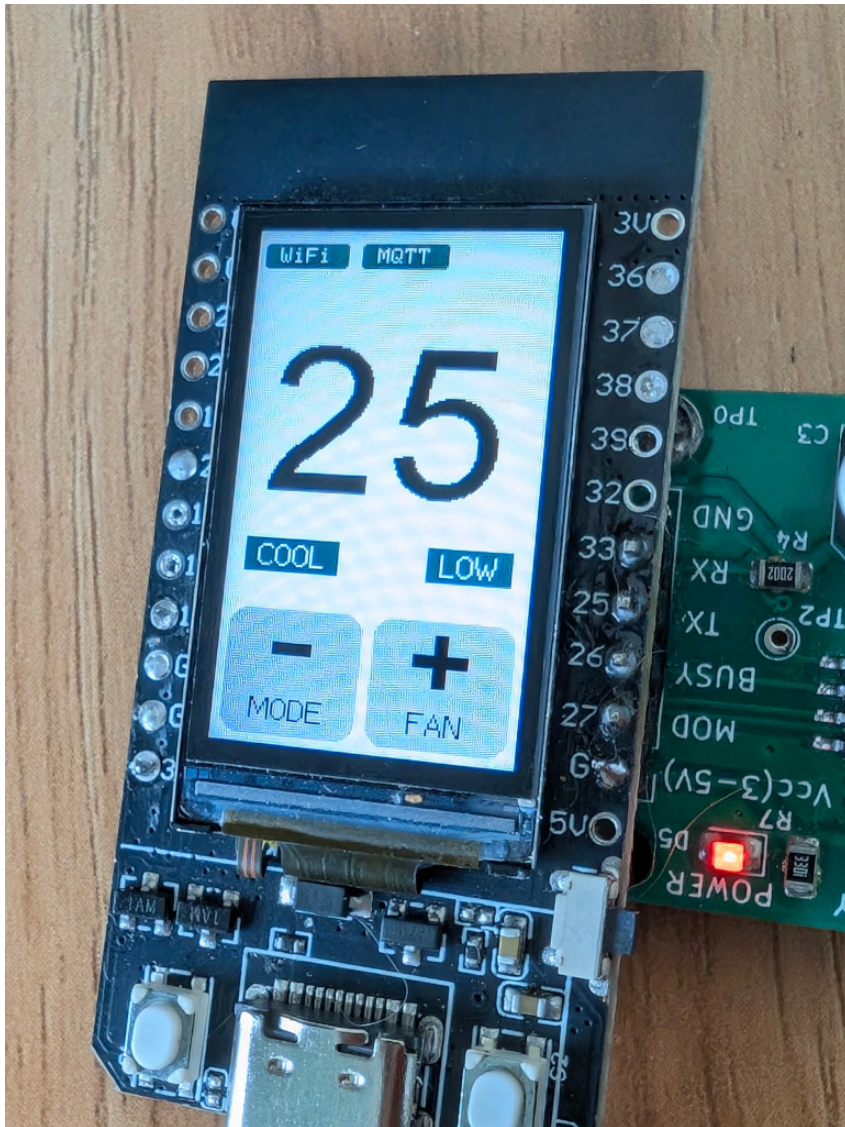
## Initialization

First, perform initialization by sampling the AC remote.





If successful, the program enters the **AC control** page:



### Button functions:

- Left short press: Decrease temperature
- Right short press: Increase temperature
- Left long press: Switch mode
- Right long press: Switch fan speed
- Double short press: Enter power control page (left = power on, right = power off)
- Double long press: Enter main menu

Whenever an action changes AC state, BC7215A transmits the corresponding IR signal, and an indicator appears at the top-right of the screen.

---

## ESP32 MQTT Version

---

The MQTT version extends the LCD version with:

1. MQTT-based network control
2. Air conditioner status reporting

It uses **Nick O’Leary’s PubSubClient library** (available in Arduino IDE Library Manager).

## 1. Preparation Before Compilation

Before compiling, uncomment the following lines in the source and replace with your own values:

```
// WiFi and device configuration - replace with your values
#define MY_WIFI_SSID      "Your WiFi Name"
#define MY_WIFI_PASSWORD "Your WiFi Password"
#define MY_UUID           "Your UUID"
```

Use a UUID generator to create a unique device ID. If two devices share the same UUID, one will be disconnected by the MQTT server.

Example UUID: `b1225e25-81c8-43d7-8183-6f5793408242`

Default MQTT broker: `broker.hivemq.com`

(You may also use public brokers such as `broker.emqx.io` ).

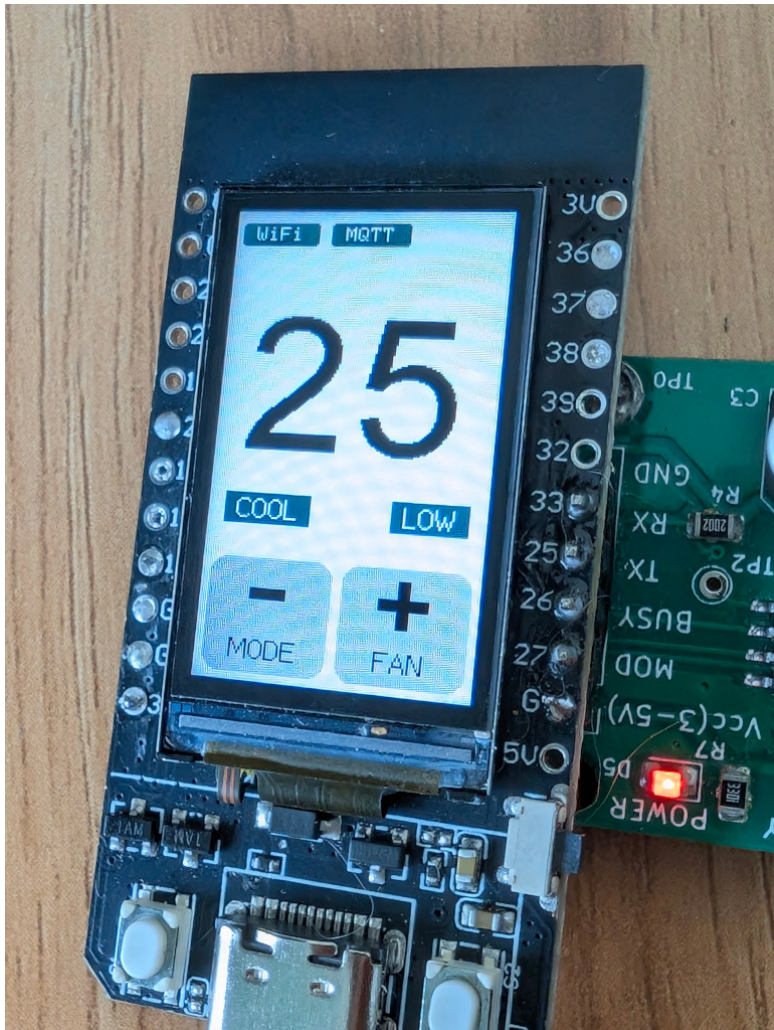
 **Note:** This demo uses an unencrypted connection. For production, use TLS-encrypted MQTT.

---

## 2. Network-Controlled A/C

On startup, the device connects to WiFi, then to the MQTT server. Once connected, WiFi and MQTT icons appear on the screen.





### MQTT Topics:

- Temperature: `BC7215A/<UUID>/var/temp`
- Mode: `BC7215A/<UUID>/var/mode`
- Fan speed: `BC7215A/<UUID>/var/fan`
- Power: `BC7215A/<UUID>/var/power`

### Example:

`BC7215A/b1225e25-81c8-43d7-8183-6f5793408242/var/temp`

### Message formats:

- **Temperature:** `"16"` – `"30"`
- **Mode:**
  - 0 = Auto
  - 1 = Cool
  - 2 = Heat
  - 3 = Dry

- 4 = Fan
  - **Fan:**
    - 0 = Auto
    - 1 = Low
    - 2 = Medium
    - 3 = High
  - **Power:**
    - 0 = Off
    - 1 = On
- 

## A/C Online Monitor

Once connected with MQTT server, any A/C status change will be published online, including both MQTT command and local operation, if program is in "parsing mode", it will also report the parsing result online. So you can monitor the A/C working status online even the user operates the A/C by a traditional IR remote.



## MQTT Client

---

Most free public brokers provide web or desktop MQTT clients. Any client can publish control messages as long as it connects to the same server and topics.

Example HiveMQ web client:

<https://www.hivemq.com/demos/websocket-client/>

To use it: open the page, leave all fields as is and click "Connect",


**HIVEMQ**
Websockets Client Showcase


**HIVEMQ** CLOUD
 

Need a fully managed MQTT broker?  
 Get your own Cloud broker and connect up to 100 devices for free.

Get your free account

**Connection**

Host

mqtt-dashboard.com

Port

8884

ClientID

clientId-sp1vpcW1Yd

Connect

Username

Password

Keep Alive

60

SSL

☐

Clean Session

☐

Last-Will Topic

Last-Will QoS

0

Last-Will Retain

☐

Last-Will Message

Publish

Subscriptions

Messages

Click Here

Leave As Is

When it's connected, setup the topics and and it's ready to go.


**HIVEMQ**
Websockets Client Showcase


**HIVEMQ** CLOUD
 

Need a fully managed MQTT broker?  
 Get your own Cloud broker and connect up to 100 devices for free.

Get your free account

**Connection**

connected

**Publish**

Topic

2-0f8c-4431-a49f-30a6d9cb3737/var/temp

QoS

0

Retain

☐

Publish

Message

20

**Subscriptions**

Add New Topic Subscription

Qos: 2

BC7215A/11741ad2...

**Messages**

2026-01-22 16:07:01

Topic: BC7215A/11741ad2-0f8c-4431...

Qos: 0

Temp=24, Mode= COOL , Fan= AUTO , Power= ON

A/C Control Topic, find in source code:

```

52 const char* TEMP_TOPIC = "BC7215A/" MY_UUID "/var/temp"; // Temperature control topic
53 const char* MODE_TOPIC = "BC7215A/" MY_UUID "/var/mode"; // AC mode control topic
54 const char* FAN_TOPIC = "BC7215A/" MY_UUID "/var/fan"; // Fan speed control topic
55 const char* POWER_TOPIC = "BC7215A/" MY_UUID "/var/power"; // Power on/off topic
56 const char* REPORT_TOPIC = "BC7215A/" MY_UUID "/var/report"; // Status reporting topic
57
  
```

Set Value

Status Report Topic

Received A/C Status Report

"Publish" area is for the online A/C control, temperature and others each have their own topics, enter the value to be set and "publish" it, the BC7215A module will send IR to control the A/C.

"Subscriptions" is for the A/C status report, received reports will be shown in "Messages" area.



---

## AC Status Reporting

Whenever an IR command is sent (via button or MQTT), or in the parsing mode when an IR signal is received, the new AC state is reported to the MQTT server. Clients subscribed to the **report topic** will receive updates:

```
BC7215A/<UUID>/var/report
```